

T.Y.B.C.A (SEM-6)

[Subject code for Practical-2611000906011002]

Course Code: 602**Course Title: Data Analytics using Python**

Course Code	602
Course Title	Data Analytics using Python
Credits	4
Course Category	Major Course
Level of Course	400-499 (Advance Level)
Teaching Hours	30 Hours of class-room teaching + 60 Hours of Applied work(Project)
Minimum Hours/ Semester	30 hours of Theory + 60 Hours of Applied work (Project) (Including class work, examination, preparation etc.)
Review / Revision	-
Implementation Year:	Only during A.Y. 2025-2026
Purpose of Course	This course aims to introduce students to data analytics techniques using Python, with a focus on Exploratory Data Analysis (EDA), regression, and supervised learning. It equips learners with practical skills in handling data, automating EDA, and applying machine learning concepts in real-world scenarios.
Course Objective	1) Explain the concepts and techniques of exploratory data analysis including handling missing data, outliers, and distribution types. 2) Apply Python libraries like Pandas and NumPy to automate and perform efficient data analysis tasks. 3) Describe the basics of regression and correlation, and understand their role in data analysis and machine learning. 4) Understand key concepts of machine learning including types, benefits, and real-world applications. 5) Evaluate supervised learning models using loss functions such as MSE and MAE, and explain concepts like overfitting and underfitting.
Course Content	Unit-1 : Fundamentals of Data Analytics : 1.1 Exploratory Data Analysis (EDA) 1.1.1 Types of Exploratory Data Analysis: 1.1.2 Univariate Analysis 1.1.3 Bivariate Analysis 1.1.4 Multivariate Analysis 1.1.5 Handling Missing Data and Outliers 1.2 Understanding the Data: 1.2.1 Quantitative Data : Discrete and Continuous 1.2.2 Qualitative Data : Non-numerical (Normal and Ordinal) 1.3 Spread of Data 1.3.1 Normal Distribution 1.3.2 Skewed Distribution 1.3.3 Skewness and Kurtosis

Unit-2 Automate EDA (Exploratory Data Analysis)**2.1 Python Libraries to Automate Exploratory Data Analysis****2.1.1 Pandas and Numpy****2.2 Regression****2.2.1 Characteristics of Regression****2.2.2 Dependent and Independent variables****2.2.3 Covariance and Correlation****2.3 Machine Learning Basics:****2.3.1 Concepts of Machine learning****2.3.1.1 Understanding machine learning****2.3.1.2 Benefit of machine learning****2.3.1.2 Machine learning life cycle****2.4 Types of Machine Learning:****2.4.1 Supervised and Unsupervised Learning****2.4.2 Applications of ML in real-world scenarios****Unit-3: Understanding Supervised Learning****3.1 Overview of Supervised Learning****3.1.1 Concepts of Supervised Learning****3.1.2 Difference between Classification and Regression****3.2 Basic Terminologies****3.2.1 Dataset, Features, Labels****3.2.2 Training Data, Test Data, validation Data****3.2.3 Overfitting, Underfitting****3.3 Loss functions :****3.3.1 Mean Squared Error (MSE)****3.3.2 Definition of MSE****3.3.2.1 Computing MSE and its properties****3.3.3 Mean Absolute Error (MAE)****3.3.3.1 Definition of MAE****3.3.3.2 Computing MAE and its properties****3.3.3.3 Understanding Regression and R^2 .**

Unit-4 : Vedic Mathematics Sutras :

- 4.1 Nikhilam Navatashcaramam Dashatah : "All from 9 and the last from 10."
- 4.2 Ekadhikena Purvena : "By one more than the previous one."
- 4.3 Udharan : "The extraction."
- 4.4 Paraavartya : "Transposition and cancellation."
- 4.5 Shunyam Saamyasamuccaye : "When the sum is the same that sum is zero."
- 4.6 Anurupyena : "Proportionately."
- 4.7 Sankalana-Vyavakalanabhyam : "By addition and by subtraction."
- 4.8 Puranapuranaabhyam : "By the completion or non-completion."
- 4.9 Chalana-Kalana : "By motion or by applying a shift."
- 4.10 Yavadunam : "Whatever is the deficiency."
- 4.11 Vyastisamanstih : "The parts and the whole."
- 4.12 Sesanyan : "The remainder."
- 4.13 Gunitasamuchyah : "The product of the sum."
- 4.14 Vistaran : "Expansion."
- 4.15 Rupan : "Form."
- 4.16 Chidana : "By splitting."

[Implementation of all sutras of Unit-4 in computer Lab. Using C / Python / Any Prog. Language]

[Students will submit E-Document for Project report.

One internal guide will be allocated for every ten groups

All groups are required to contact their internal guides once a week to endorse their project progress work.] [Students need to complete their project by 10th of February/15th of August(Winter session)]

[All Units carry Equal Weightage]



What is EDA (Exploratory Data Analysis)?

Exploratory Data Analysis (EDA) is the process of analyzing and summarizing datasets before applying any machine learning models. It helps to understand:

- **What the data looks like**
- **What patterns or trends it may have**
- **If there are any missing values, outliers, or errors**
- **The relationship between variables**

 *EDA is like getting to know data before using it for prediction or decision-making.*

Types of EDA

1. Quantitative EDA (Statistical)

This involves using numerical summaries to describe the data.

Example:

For the column Age in a dataset:

- Mean = 35.2
- Median = 34
- Mode = 30
- Standard Deviation = 5.6

These values tell you the central tendency and spread of the data.

2. Graphical EDA (Visual)

This involves using plots and charts to visualize the data for better understanding.

Common charts:

- **Histogram: Distribution of one variable**
- **Boxplot: Spread and outliers**
- **Scatter Plot: Relationship between two variables**
- **Heatmap: Correlation between many variables**

Example:

A boxplot of salary data shows that most people earn between ₹30,000 to ₹60,000, but a few earn above ₹1,00,000 (outliers).



3. Univariate Analysis

- **Study of a single variable**
- **Helps in understanding distribution, central tendency, spread**

Example:

For the variable Marks of students:

- **Plot a histogram to see how marks are distributed**
- **Find average marks (mean), and variation**

✓ 4. Bivariate Analysis

- Study of two variables together
- Shows relationships, trends, and correlations

Example:

Compare Study Hours and Marks:

- Scatter plot shows that more study hours generally lead to higher marks
- Correlation coefficient might be 0.85 → strong positive relation

✓ 5. Multivariate Analysis

- Study of more than two variables
- Used to find complex patterns or interactions

Example:

Using Age, Study Hours, and Marks together to predict student performance.

- Use a heatmap to show correlations

- A pairplot shows all possible combinations visually

Summary Table

Type	Description	Example
Quantitative	Statistical summary	Mean, Median, Std. Dev. of Salary
Graphical	Visual plots	Boxplot of Age, Histogram of Sales
Univariate	Single variable analysis	Frequency of marks
Bivariate	Two-variable relation	Study Hours vs Marks (scatter plot)
Multivariate	Multiple variables	Age + Marks + Study Hours (heatmap)

Topic 1.3.1: Types of Exploratory Data Analysis (EDA)

EDA (Exploratory Data Analysis) is the process of examining the data to understand its main characteristics before applying any model.

Types of EDA:

EDA can be broadly classified into the following types:

1. **Descriptive Statistics** – Summarizing data using mean, median, mode, etc.
2. **Data Visualization** – Using plots to observe patterns and trends.
3. **Outlier Detection** – Finding data points that are significantly different.
4. **Missing Value Analysis** – Identifying gaps in the data.
5. **Correlation Analysis** – Understanding relationships between features.



Practical Problem Using Python

Let's use a small dataset of **students' performance** to explore different types of EDA.

We'll create a sample dataset inline using Python.

Step 1: Setup

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# For Jupyter: optional
%matplotlib inline
```

Step 2: Create Sample Dataset

```
data = {
    'Student': ['A', 'B', 'C', 'D', 'E',
                'F', 'G', 'H'],
    'Maths': [78, 85, 62, 90, 70, 75,
              np.nan, 88],
    'Science': [72, 89, 65, 95, 67,
                np.nan, 74, 91],
    'English': [80, 83, 78, 85, 72, 79,
                77, 84]
}
```

```
df = pd.DataFrame(data)
df
```

output :

	Student	Maths	Science	English
0	A	78.0	72.0	80
1	B	85.0	89.0	83
2	C	62.0	65.0	78
3	D	90.0	95.0	85
4	E	70.0	67.0	72
5	F	75.0	NaN	79
6	G	NaN	74.0	77
7	H	88.0	91.0	84



Dr. Snehal K Joshi

1. Descriptive Statistics

```
# Basic descriptive statistics  
df.describe()
```

	Maths	Science	English
count	7.000000	7.000000	8.00000
mean	78.285714	79.000000	79.75000
std	10.177005	12.342339	4.26782
min	62.000000	65.000000	72.00000
25%	72.500000	69.500000	77.75000
50%	78.000000	74.000000	79.50000
75%	86.500000	90.000000	83.25000
max	90.000000	95.000000	85.00000



Descriptive Statistics (in EDA)

Descriptive statistics is a method to **summarize and describe** the main features of a dataset using numbers.

It helps us understand the basic structure and patterns in the data.

Key terms:

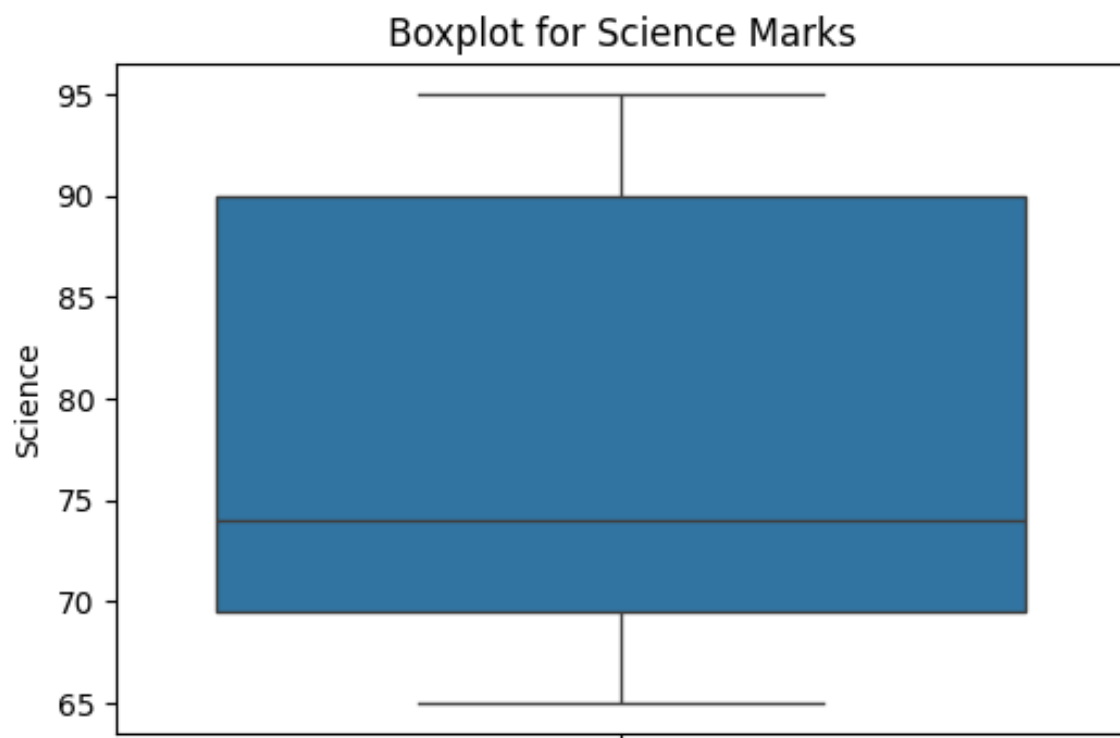
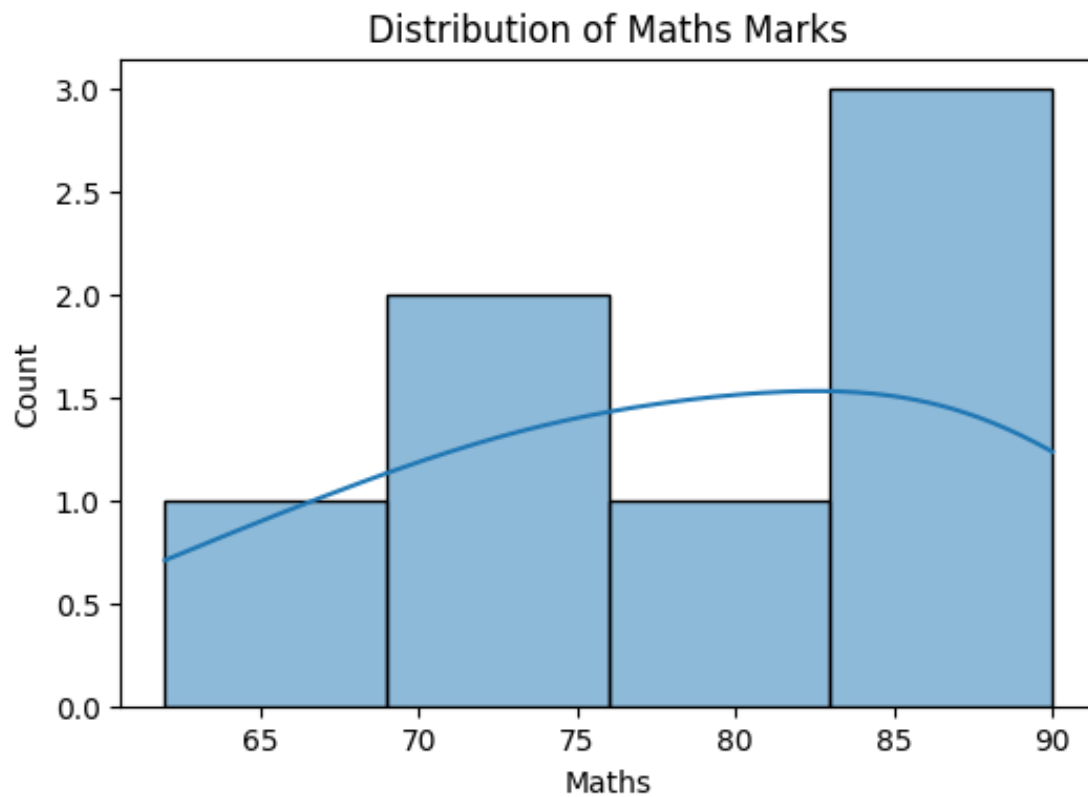
- **Mean:** The average value.
- **Median:** The middle value when data is sorted.
- **Mode:** The value that appears most often.

- **Standard Deviation (std):** How much the values vary from the mean.
- **Min & Max:** The smallest and largest values.
- **Quartiles (25%, 50%, 75%):** Values that divide the data into four equal parts.

✦ These help us get a **quick summary** of each column in a dataset.

2. Data Visualization

```
# Histogram for Maths scores
plt.figure(figsize=(6,4))
sns.histplot(df['Maths'], kde=True)
plt.title("Distribution of Maths Marks")
plt.show()
# Box plot for Science
plt.figure(figsize=(6,4))
sns.boxplot(y=df['Science'])
plt.title("Boxplot for Science Marks")
plt.show()
```

Output:

Histogram

- A **histogram** is a **bar graph** that shows how data is **distributed**.
- It groups data into **ranges (called bins)** and shows how many values fall in each range.
- Useful for understanding:
 - **Frequency** of values
 - **Shape** of distribution (normal, skewed, etc.)

Example: Histogram of marks shows how many students scored between 60–70, 70–80, etc.

Boxplot (Box-and-Whisker Plot)

- A **boxplot** shows the **spread and outliers** in the data.
- It displays:
 - **Minimum**
 - **1st Quartile (25%)**
 - **Median (50%)**
 - **3rd Quartile (75%)**
 - **Maximum**
- **Dots or stars** show **outliers** (unusual values).

Example: Boxplot of marks helps to quickly spot if any student scored much lower or higher than others.

✕ 3. Handling Missing Data

```
# Check missing values
df.isnull().sum()
# Fill missing values with mean
df_filled =
df.fillna(df.mean(numeric_only=True))
df_filled
```

Notes by:

	Student	Maths	Science	English
0	A	78.000000	72.0	80
1	B	85.000000	89.0	83
2	C	62.000000	65.0	78
3	D	90.000000	95.0	85
4	E	70.000000	67.0	72
5	F	75.000000	79.0	79
6	G	78.285714	74.0	77
7	H	88.000000	91.0	84

✓ Code Explanation

🔧 Handling Missing Data in EDA

Sometimes, datasets have **missing values**. These are shown as NaN (Not a Number) in Python. We must handle them before analysis or modeling.

⚡ *Line 1: Check missing values*

```
df.isnull().sum()
```

- `df.isnull()` → returns a **True/False** table showing where values are missing.
- `.sum()` → counts how many True values (i.e., missing values) are in each column.
- ✦ **Result:** Tells that **how many missing values** are present in each column.

⚡ *Line 2: Fill missing values with mean*

```
df_filled =  
df.fillna(df.mean(numeric_only=True))
```

- `df.mean(numeric_only=True)` → calculates the **mean of each numeric column** (ignores text columns like "Student").

- `df.fillna(...)` → replaces each missing value (NaN) with the **mean value** of its respective column.
- The result is saved in a new DataFrame called `df_filled`.

📌 So, **all missing values** in numeric columns are replaced with the **average of that column**.

🔗 *Line 3: Display the new dataset*

`df_filled`

- Shows the updated dataset **with no missing values**.

🔄 Example Before and After:

Suppose you had:

Student Maths Science

A 85 NaN

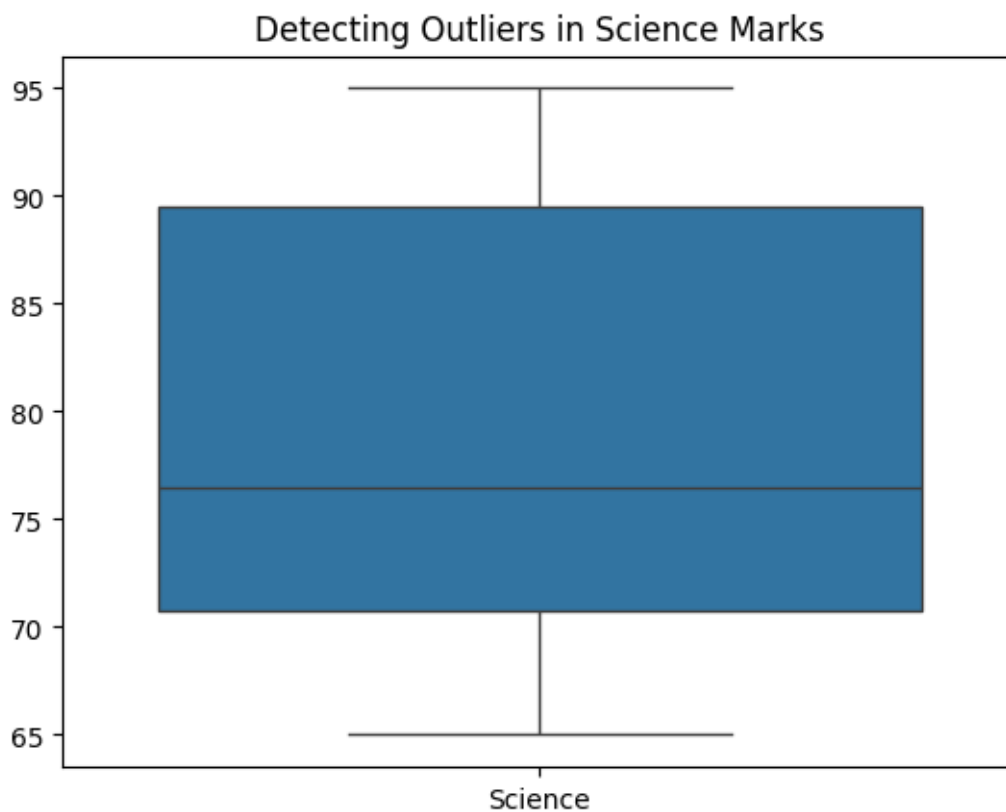
B NaN 80

- Mean of Maths = 85 → replaces NaN in Maths
- Mean of Science = 80 → replaces NaN in Science

4. Outlier Detection

Let's visually inspect outliers in Science

```
sns.boxplot(data=df_filled[['Science']])  
plt.title("Detecting Outliers in Science Marks")  
plt.show()
```



What is an Outlier?

An **outlier** is a **data point that is very different** from the rest of the values in a dataset.

It lies **far away from the average or normal range**, and can affect the results of analysis or models.



Example:

Suppose these are marks in Maths:

[70, 72, 74, 75, 78, 80, 150]

- Most students scored around 70–80.
- But **150** is **much higher** than others → this is an **outlier**.



How to detect outliers?

We often use a **boxplot** to detect outliers:

- Points **outside the box's whiskers** are shown as **dots or stars** → these are outliers.

? Why do outliers matter?

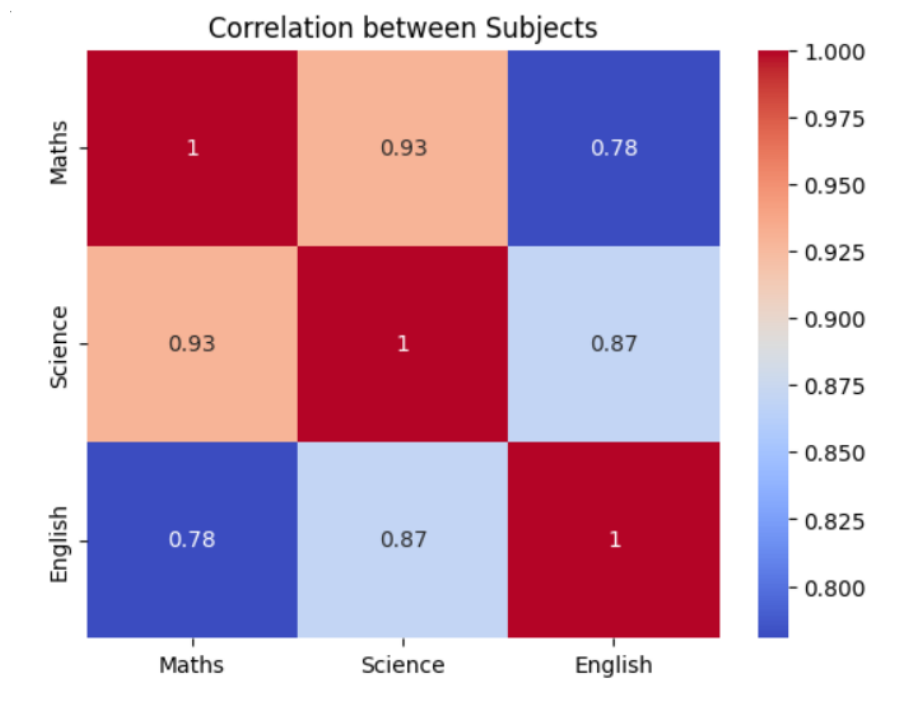
- Outliers can **mislead analysis** or affect **model accuracy**.
- They may be:
 - **Real** (e.g., a top-performing student)
 - **Errors** (e.g., wrong data entry)

So, we either **investigate**, **remove**, or **treat** them depending on the situation.

5. Correlation Analysis

```
# Correlation matrix
correlation =
df_filled.corr(numeric_only=True)
sns.heatmap(correlation,
annot=True, cmap="coolwarm")
plt.title("Correlation between
Subjects")
plt.show()
```

OUTPUT:



∞ What is Correlation?

- **Correlation** tells us how **strongly two variables are related** to each other.
- It ranges from:
 - **+1** → strong positive relationship (both go up together)
 - **0** → no relationship
 - **-1** → strong negative relationship (one goes up, other goes down)

Example: If students who score high in Maths also score high in Science → **high positive correlation**.

✓ Code Explanation

```
correlation =  
df_filled.corr(numeric_only=True)
```

- This line calculates the **correlation matrix** between **numeric columns** only (e.g., Maths, Science, English).
- `df_filled` is your cleaned dataset (no missing values).
- `correlation` will be a **table showing the correlation** between each pair of subjects.

```
sns.heatmap(correlation, annot=True,  
cmap="coolwarm")
```

- `sns.heatmap(...)` → draws a **color-coded table** (heatmap).
- `annot=True` → shows the **correlation values** inside the boxes.
- `cmap="coolwarm"` → color scale from **blue (low)** to **red (high)**.

```
plt.title("Correlation between Subjects")  
plt.show()
```

✦ The Outcome:

- Visually compare how different subjects are related.
- High values like **0.95** mean strong positive correlation.
- Near-zero values mean no strong relation.

1.3 Spread of Data

What is Spread of Data?

- Spread of data tells us how much the data values vary or are scattered.
- It shows whether data values are **close together or far apart**.
- Spread helps us understand **consistency and reliability** of data.

Example:

- Marks: 70, 71, 72, 73 → small spread
- Marks: 30, 50, 70, 90 → large spread

1.3.1 Normal Distribution

What is Normal Distribution?

- A **normal distribution** is a **symmetrical, bell-shaped curve**.
- Most values are near the **average (mean)**.
- Mean = Median = Mode

Key Features:

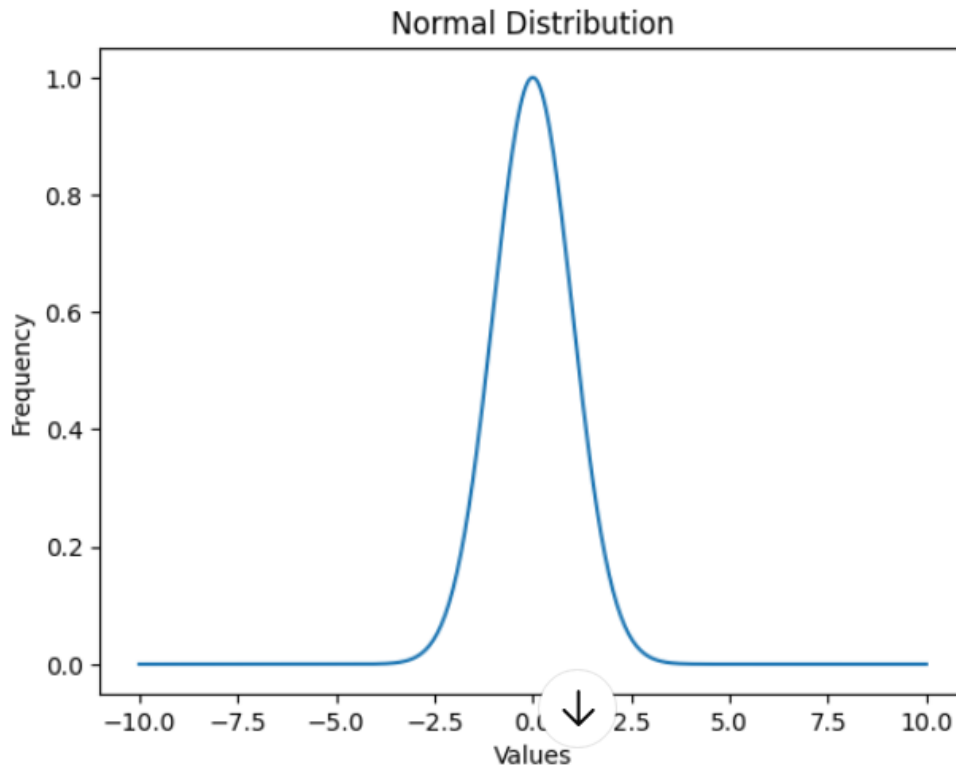
- Left side = Right side (perfect balance)
- Maximum data is in the middle
- Fewer values at extremes

Example:

- Height of students in a class
- IQ scores
- Exam marks of a large group

 *Real-life idea:*

Most students score average marks, few score very high or very low.



Dr. Snehal K Joshi

1.3.2 Skewed Distribution

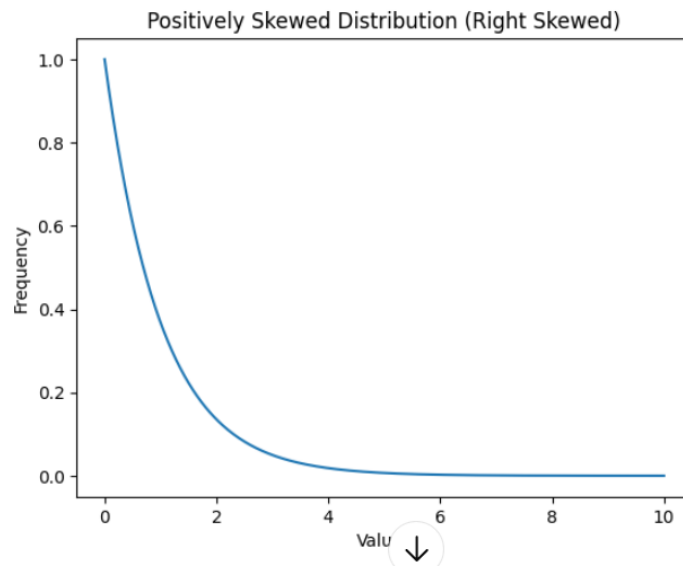
What is Skewed Distribution?

- When data is **not symmetrical**.
- One side has a **longer tail** than the other.

Types of Skewness:

1. Positively Skewed (Right Skewed)

- Tail is longer on the **right side**
- Mean > Median > Mode

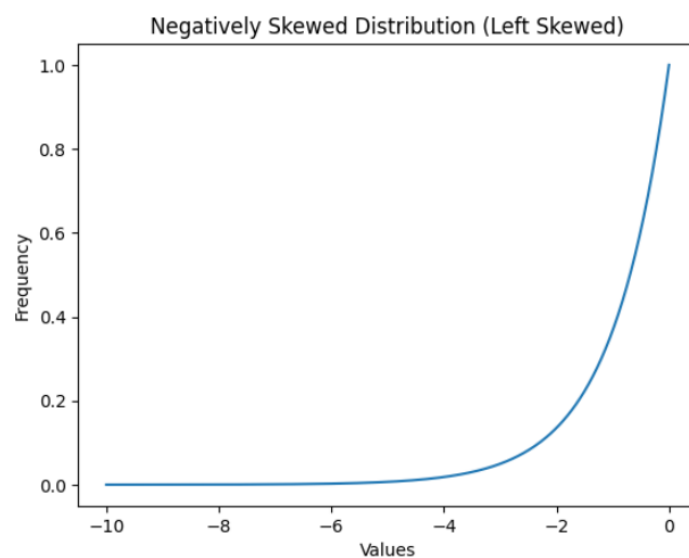


Example:

- Income distribution
(Most people earn low income, few earn very high)

2. Negatively Skewed (Left Skewed)

- Tail is longer on the **left side**
- $\text{Mean} < \text{Median} < \text{Mode}$



Example:

- Easy exam marks
(Most students score high, few score very low)

1.3.3 Skewness and Kurtosis

Skewness

- Measures **asymmetry** of data.
- Tells whether data is:
 - Symmetrical (Normal)
 - Right skewed
 - Left skewed

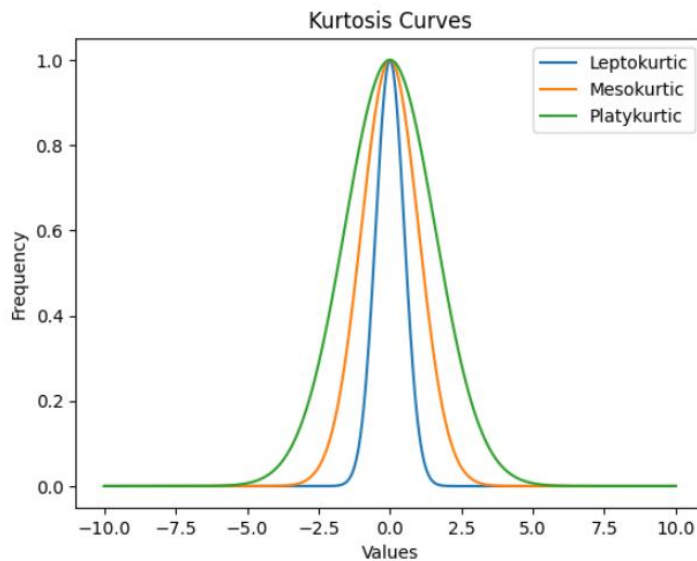


Easy way to remember:

- Right tail longer → Positive skew
- Left tail longer → Negative skew

Kurtosis

- Measures **peakedness or flatness** of data.
- Shows how data is **concentrated around the mean**.



Types of Kurtosis:

1. Leptokurtic

- Very **high peak**
- Data tightly clustered
- Example: Consistent exam marks

2. Mesokurtic

- Normal peak
- Normal distribution

3. Platykurtic

- Flat curve
- Data spread widely
- Example: Marks spread across all ranges

Quick Summary :

Concept	Meaning
Spread	How scattered the data is
Normal Distribution	Symmetrical bell shape
Skewness	Direction of data spread
Kurtosis	Height or flatness of curve

Notes by:
Dr. Snehal K Joshi

Exercise



EDA Practical Homework Assignments

◆ Practical 1: Load and Explore a Dataset

- Load any CSV dataset (e.g., Titanic or Iris).
- Display the first 5 and last 5 rows.
- Show basic info using `.info()` and `.describe()`.
- Count missing values column-wise.

◆ Practical 2: Univariate Analysis

- Choose one **numerical** and one **categorical** column.
- Perform descriptive statistics (mean, median, mode, std).
- Draw:
 - Histogram
 - Box plot
 - Pie chart (for categorical column)

◆ Practical 3: Bivariate Analysis

- Select two columns (e.g., Age and Survival, or PetalLength and PetalWidth)
- Plot:
 - Scatter plot
 - Correlation matrix
 - Bar chart (for one numerical and one categorical)

◆ Practical 4: Multivariate Analysis

- Use **pairplot** and **heatmap** to visualize relationships between multiple features.
- Interpret at least two insights based on the heatmap.

◆ Practical 5: Handling Missing Data

- Identify columns with missing values.
- Fill missing values using:
 - Mean (for numeric columns)
 - Mode (for categorical columns)
 - Interpolation
- Compare dataset before and after filling.

◆ Practical 6: Detecting and Handling Outliers

- Use boxplots to detect outliers in any one numerical column.
- Use IQR method to identify the outlier threshold.
- Remove or cap the outliers.
- Show data before and after cleaning.

◆ Practical 7: Mini-EDA Report

- Choose any dataset (small and public).
- Write a 1-page EDA report covering:
 - Dataset summary
 - At least 2 plots
 - Missing value handling
 - Insights from bivariate/multivariate analysis

1.2 Understanding the Data

Data is a collection of facts, values, or observations used for analysis and decision-making.

Broadly, data is classified into **Quantitative** and **Qualitative** data.

1.2.1 Quantitative Data

Quantitative data is **numerical data** that can be measured or counted.

It answers “**how much?**” or “**how many?**”

Notes by:

Dr. Snehal K Joshi

Types of Quantitative Data

a) Discrete Data

- Data that can take **only specific, separate values**
- Usually **countable**
- No fractional values

Examples:

- Number of students in a class (30, 31, 32)
- Number of cars in a parking lot
- Number of defects in a product

Key Points:

- Whole numbers

- Cannot be divided into smaller parts meaningfully

b) Continuous Data

- Data that can take **any value within a given range**
- Measured, not counted
- Can include **decimals and fractions**

Examples:

- Height (165.5 cm)
- Weight (62.75 kg)
- Time (2.35 hours)
- Temperature (36.6°C)

Notes by:

Dr. Snehal K Joshi

Key Points:

- Infinite possible values
- More precise measurements

1.2.2 Qualitative Data

Qualitative data is **non-numerical data** that describes **qualities or characteristics**.

It answers “**what type?**” or “**which category?**”

Types of Qualitative Data

a) Nominal Data

- Categories **without any order**
- Only used for labeling or naming

Examples:

- Gender (Male, Female)
- Blood group (A, B, AB, O)
- City names
- Product type

Key Points:

- No ranking
- Cannot be mathematically compared

b) Ordinal Data

- Categories **with a meaningful order**
- Difference between values is **not measurable**

Examples:

- Customer satisfaction (Poor, Average, Good, Excellent)
- Education level (School, Graduate, Post-Graduate)
- Ratings (1 star, 2 star, 3 star)

Key Points:

- Order matters
- Exact difference between categories is unknown

1.3 Spread of Data

Spread of data shows **how data values are distributed or dispersed** around a central value.

Common measures of spread:

- Range
- Variance
- Standard Deviation

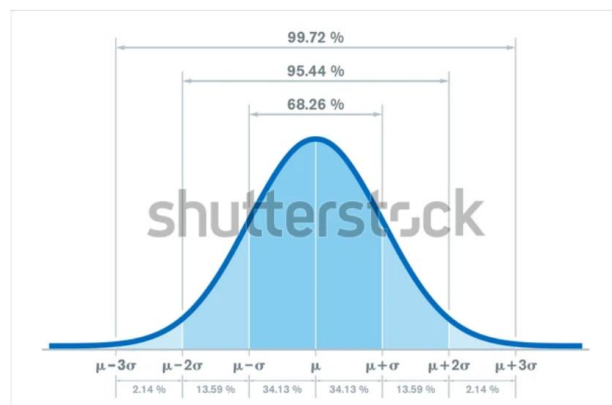
Understanding distribution helps in identifying **patterns, trends, and outliers**.

1.3.1 Normal Distribution

A normal distribution is a **symmetrical, bell-shaped curve**.

Characteristics:

- Mean = Median = Mode
- Data is evenly spread on both sides
- Most values lie near the center

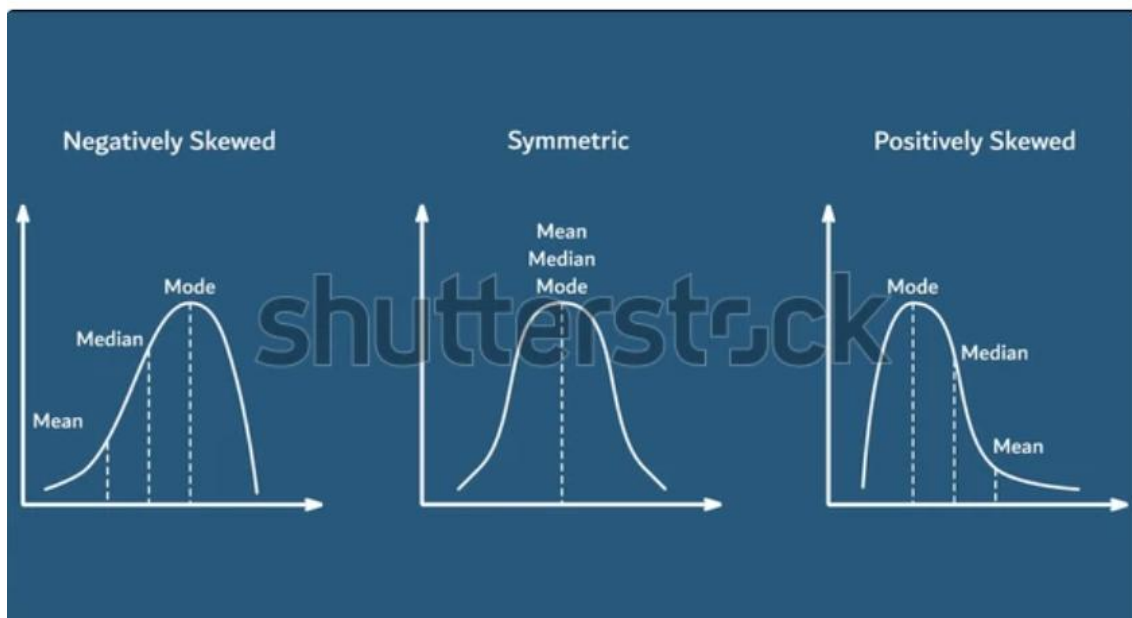


Examples:

- Heights of students
- Exam marks (in large groups)
- Measurement errors

Importance:

- Many statistical methods assume normal distribution
- Easy to analyze and predict outcomes



1.3.2 Skewed Distribution

A skewed distribution is **not symmetrical**.

Types of Skewness

a) Positive Skew (Right-Skewed)

- Long tail on the **right side**
- $\text{Mean} > \text{Median} > \text{Mode}$

Examples:

- Income distribution
- House prices

b) Negative Skew (Left-Skewed)

- Long tail on the **left side**
- $\text{Mean} < \text{Median} < \text{Mode}$

Examples:

- Difficult exam scores
- Retirement age data

1.3.3 Skewness and Kurtosis**Skewness**

Skewness measures the **asymmetry** of the data distribution.

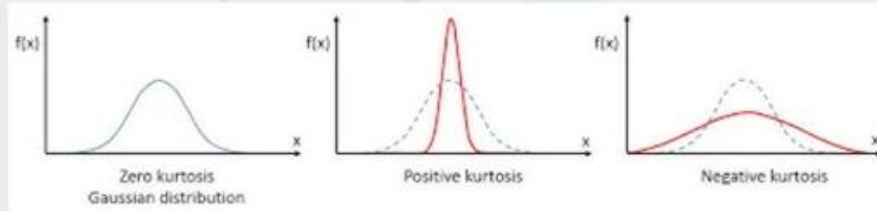
Skewness Value	Meaning
0	Symmetrical distribution
> 0	Positively skewed
< 0	Negatively skewed

Kurtosis

Kurtosis measures the **peakedness or flatness** of a distribution.

Kurtosis

- Kurtosis is a measure of tailedness
- Kurtosis is a descriptive statistic that describes the shape of the distribution.



Types of Kurtosis

1. Mesokurtic

- Normal peak
- Example: Normal distribution

2. Leptokurtic

- High, sharp peak
- More outliers

3. Platykurtic

- Flat peak
- Fewer outliers

Summary

- **Quantitative data** is numerical (Discrete & Continuous)

- **Qualitative data** is categorical (Nominal & Ordinal)
- **Spread of data** explains variability
- **Normal distribution** is symmetric
- **Skewness** shows direction of spread
- **Kurtosis** shows sharpness of the curve



1.3.6 Detecting and Handling Outliers



Understand an Outlier:

An **outlier** is a data point that is **significantly different** from the rest of the values.

Outliers can:

- Mislead analysis or visualization
- Skew statistical metrics (mean, standard deviation)
- Affect machine learning model performance

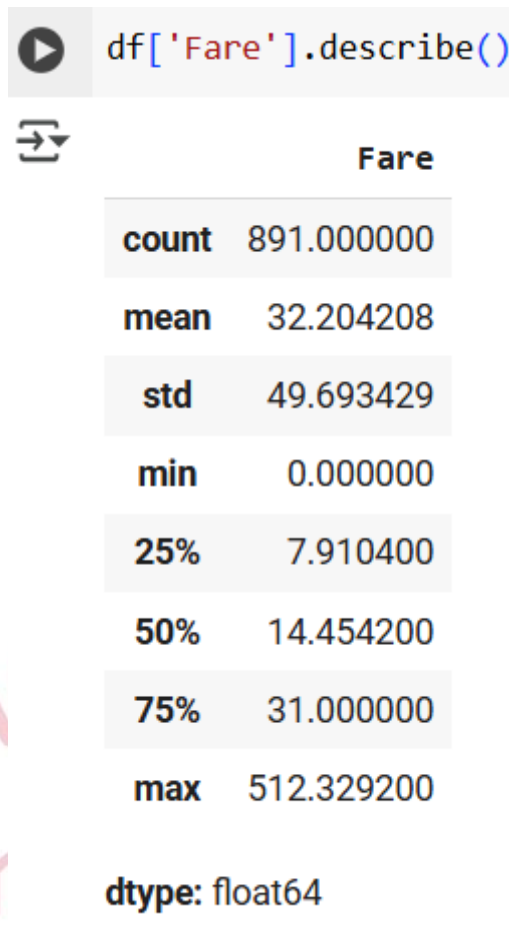


Detecting Outliers:

◆ 1. Using Descriptive Statistics

```
df['Fare'].describe()
```


Output:



The image shows a Jupyter Notebook interface. At the top, there is a code cell with a play button icon and the code `df['Fare'].describe()`. Below the code cell is an output cell containing a table of statistics for the 'Fare' column. The table has a header 'Fare' and rows for 'count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max', and 'dtype: float64'.

	Fare
count	891.000000
mean	32.204208
std	49.693429
min	0.000000
25%	7.910400
50%	14.454200
75%	31.000000
max	512.329200
dtype:	float64

Check:

- Extremely high **max value**
- Large **difference between mean and median**

◆ 2. Using Boxplot

```
import seaborn as sns
```

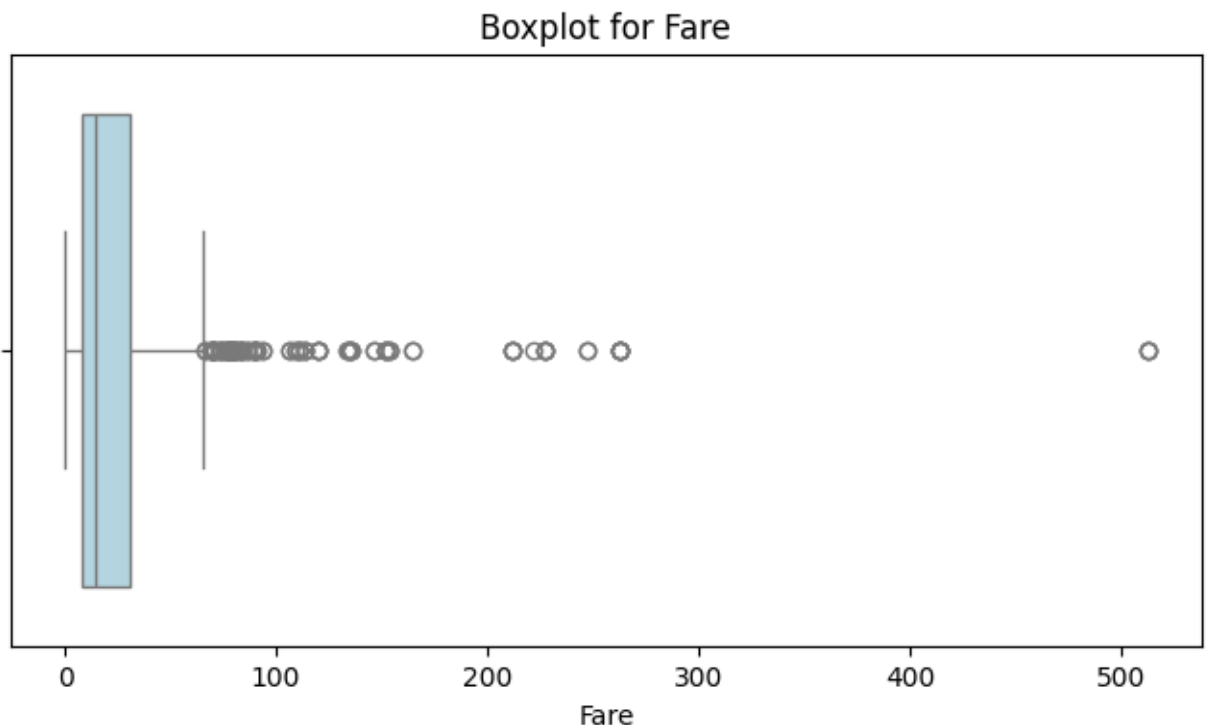
```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 4))
```

```
sns.boxplot(x=df['Fare'], color='lightblue')  
plt.title("Boxplot for Fare")  
plt.show()
```

🎯 Any data point outside the whiskers is likely an outlier.

Output:



◆ 3. Using IQR Method (Interquartile Range)

```

▶ Q1 = df['Fare'].quantile(0.25)
  Q3 = df['Fare'].quantile(0.75)
  IQR = Q3 - Q1
  # Define bounds
  lower = Q1 - 1.5 * IQR
  upper = Q3 + 1.5 * IQR
  # Filter outliers
  outliers = df[(df['Fare'] < lower) | (df['Fare'] > upper)]
  print("Number of outliers in Fare:", len(outliers))

```

➡ Number of outliers in Fare: 116

✓ Handling Outliers:

Method	When to Use
Remove outliers	If outliers are data entry errors
Cap values (clipping)	If they are extreme but real
Transform data	Use log or square root to reduce skew

◆ Example: Cap the Fare Outliers

```

# Cap fare to 95th percentile
cap_value = df['Fare'].quantile(0.95)
df['Fare'] = df['Fare'].apply(lambda x: cap_value if
x > cap_value else x)

```

✓ Summary

- Outliers **must be detected** to ensure clean, reliable models.
- You can **visualize (boxplot), detect (IQR), and handle (remove/cap/transform)** them.

Notes by:
Dr. Snehal K Joshi